

8-bit ALU

1. Specification

The ALU operates on two signed 8-bit operands **A** and **B**, selects one of nine operations through a 4-bit control input `alu_ctrl`, and produces a signed 8-bit `result` together with status flags.

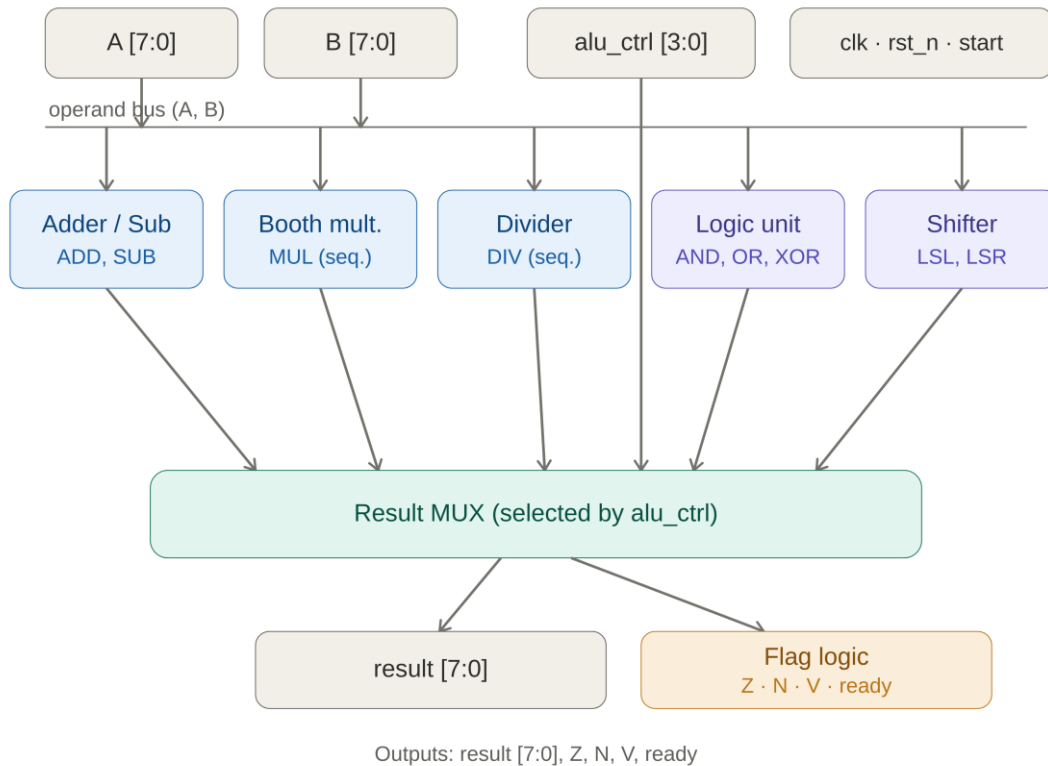
Interface signals

- **A, B** - two signed 8-bit input operands.
- **alu_ctrl [3:0]** - selects the operation (see opcode table).
- **start, clk, rst_n** - launch pulse for sequential ops, clock and active-low reset.
- **result [7:0]** - signed 8-bit output of the selected operation.
- **Z, N, V** - status flags: Zero (result is 0), Negative (MSB = 1), Overflow (signed arithmetic overflow).
- **ready** - high when the result is valid (always high for combinational ops; pulses on completion for MUL/DIV).

Operations and opcodes

alu_ctrl	Operation	Type
0 (ADD)	Addition $A + B$	combinational
1 (SUB)	Subtraction $A - B$	combinational
2 (MUL)	Multiplication $A \times B$	sequential (Booth)
3 (DIV)	Division A / B	sequential (restoring)
4 (AND)	Bitwise $A \& B$	combinational
5 (OR)	Bitwise $A B$	combinational
6 (XOR)	Bitwise $A \wedge B$	combinational
7 (LSL)	Left shift $A \ll B[2:0]$	combinational
8 (LSR)	Right shift $A \gg B[2:0]$	combinational

2. Top-level architecture



ALU datapath: operands and control feed parallel functional units, a result mux and the flag logic.

The operands **A** and **B** are distributed to all five functional units, which work in parallel. The combinational units (adder/subtractor, logic unit, shifter) produce their result in the same cycle (**ready** = 1 immediately); the sequential units (the Booth multiplier and the divider) take several cycles and signal completion through **ready**. The result multiplexer picks the output of the unit selected by **alu_ctrl**, and the flag logic derives **Z**, **N** and **V** from the result - valid only while **ready** is high.

Addition and subtraction share one 8-bit adder: for subtraction the ALU feeds $\sim B$ and sets the carry-in to 1, computing $A + (\sim B) + 1 = A - B$ in two's complement.



as a model for the α - β transition. A β - β transition

- After the optional add/subtract, the whole $\{A, C, C+1\}$ register is arithmetically shifted right by

A control unit FCM sequences the datapath through its states, driving on 8-bit control bus

TABLE	LOAD M	LOAD Q	SCAN	SUITE	CHECK
1	1	1	1	1	1
2	1	1	1	1	1
3	1	1	1	1	1
4	1	1	1	1	1
5	1	1	1	1	1
6	1	1	1	1	1
7	1	1	1	1	1
8	1	1	1	1	1
9	1	1	1	1	1
10	1	1	1	1	1
11	1	1	1	1	1
12	1	1	1	1	1
13	1	1	1	1	1
14	1	1	1	1	1
15	1	1	1	1	1
16	1	1	1	1	1
17	1	1	1	1	1
18	1	1	1	1	1
19	1	1	1	1	1
20	1	1	1	1	1
21	1	1	1	1	1
22	1	1	1	1	1
23	1	1	1	1	1
24	1	1	1	1	1
25	1	1	1	1	1
26	1	1	1	1	1
27	1	1	1	1	1
28	1	1	1	1	1
29	1	1	1	1	1
30	1	1	1	1	1
31	1	1	1	1	1
32	1	1	1	1	1
33	1	1	1	1	1
34	1	1	1	1	1
35	1	1	1	1	1
36	1	1	1	1	1
37	1	1	1	1	1
38	1	1	1	1	1
39	1	1	1	1	1
40	1	1	1	1	1
41	1	1	1	1	1
42	1	1	1	1	1
43	1	1	1	1	1
44	1	1	1	1	1
45	1	1	1	1	1
46	1	1	1	1	1
47	1	1	1	1	1
48	1	1	1	1	1
49	1	1	1	1	1
50	1	1	1	1	1
51	1	1	1	1	1
52	1	1	1	1	1
53	1	1	1	1	1
54	1	1	1	1	1
55	1	1	1	1	1
56	1	1	1	1	1
57	1	1	1	1	1
58	1	1	1	1	1
59	1	1	1	1	1
60	1	1	1	1	1
61	1	1	1	1	1
62	1	1	1	1	1
63	1	1	1	1	1
64	1	1	1	1	1
65	1	1	1	1	1
66	1	1	1	1	1
67	1	1	1	1	1
68	1	1	1	1	1
69	1	1	1	1	1
70	1	1	1	1	1
71	1	1	1	1	1
72	1	1	1	1	1
73	1	1	1	1	1
74	1	1	1	1	1
75	1	1	1	1	1
76	1	1	1	1	1
77	1	1	1	1	1
78	1	1	1	1	1
79	1	1	1	1	1
80	1	1	1	1	1
81	1	1	1	1	

4. Restoring divider

Division uses an 8-cycle **restoring division** algorithm on the operand magnitudes. The divisor loads into M, the dividend into Q, and A starts at zero. On each iteration the pair {A, Q} is shifted left by one, then M is subtracted from A:

- if the subtraction does not borrow ($A \geq M$), keep the difference and set the new quotient bit to 1;
- otherwise restore the previous A (no subtraction) and leave the quotient bit 0.

After 8 iterations Q holds the quotient and A the remainder. The ALU computes the result on the magnitudes and then applies the sign: the quotient is negated when the operand signs differ ($A[7] \neq B[7]$). Division by zero is handled as a special case (quotient forced to 0xFF, done asserted immediately).